

stable-fips203-mlkem-kem-decaps-k4

FIPS 203 Alg.21 ML-KEM-1024 KEM Decaps 实现方案 (k=4)

ascendc 工程 (定型交付)

2026-07-20

摘要

本文档描述 **定型交付算子** `examples/stable/stable-fips203-mlkem-kem-decaps-k4/` 在 Atlas A2 / Ascend910B4 上以 AscendC 实现 FIPS 203 **Algorithm 21** ML-KEM.Decaps() (参数集 `ml_kem_1024`, $K = 4$), 经 **Alg.18** Decaps_internal 串联 **Alg.15** K-PKE.Decrypt 与 **Alg.14** K-PKE.Encrypt, 并在设备侧完成 $G(m' || h)$ 、密文比对与 FO 选路后写出共享秘密 K 。

晋级 (2026-07-20): 自 `examples/stable/stable-fips203-mlkem-kem-decaps-k4/` 复制晋级 (# 交付 #); 预研副本保留。登记表: `docs/specs/fips203-mlkem1024-kem-decaps-baseline-registry.md`。

行为对照 (只读; 禁止把探针当编译依赖): `ascendc-tests/pass-fix-f203-alg21-kem-decaps-device-k4/` (CPU+SIM PASS; $K_{\max}=0$; SIM 全链 D **286803**+E **745925**; 单库 + 默认 `decaps_1session`; E3 REJECT + liboqs 分项 KAT CPU $\times 10$ +SIM $\times 3$ PASS)。Phase-D 行为对齐 `examples/stable/stable-fips203-mlkem-pke-decrypt-k4/`; Phase-E 行为对齐 `examples/stable/stable-fips203-mlkem-pke-encrypt-k4/` 及 Encaps 的「G 并入 prep」范式。写法对齐 `exp-fips203-mlkem-kem-encaps-k4-实现方案-customspec.tex`、`pass-fix-f203-alg21-kem-decaps-device-k4/INTEGRATION_PLAN.md`、`docs/notes/F203-KEM-Alg21-Decaps` 设备全链与SIM单session技术总结.md。

生产 I/O (强制; Alg.18 工程形态):

- **输入**: `input/dk_kem.bin` (3168 B) + `input/c.bin` (1568 B) + Decrypt/Encrypt 静态 LUT (与消息无关)。
- **输出**: 仅 `K.bin` (32 B)。
- m' 、 K' 、 r' (coins)、 c' 、 h 、 z 等仅 device UB/workspace; **禁止** Host 预填 r' ; **禁止** 将 $m'/K'/r'/c'/h/z$ 等中间态作为交付契约落盘 (调试 dump 须标非默认)。

设备 Launch (强制; 与 pass-fix 同拓扑): SIM 恰好 4 次 (`f203_decrypt_device_fused` $\rightarrow$ `f203_kem_dec_phase_e_prep` $\rightarrow$ `f203_encrypt_118_119` $\rightarrow$ `f203_kem_dec_fo_only`); CPU tikicpu **6 次** (Decrypt fused $\times 1$ + Phase-E: `prep` / `ntt_y` / `at_jp` / `intt_e1` / `pack_fo`)。首版允许 SIM 过渡独立 `fo_only` (与 pass-fix 一致); **工程债**: 后续可将 FO 收回 118_119/pack 尾使 SIM 降为 3 launch, 非本规格写码首版门禁。**禁止** Host memcmp/SHA3 冒充生产 FO; **禁止** 为 G 再开与 Encrypt 无关的独立产品化 launch。

自包含原则 (incubating 强制; 与 device 探针 CMake 策略不同): AscendC / Host 源码 **全部 vendored** 于本目录 (须同时 vendor Decrypt + Encrypt 可达树); **唯一允许的外部代码依赖**: `library/shared/`。**禁止**编译期长期 `#include ascendc-tests/`、其它 `examples/`、`frozen/`; 写码阶段从 stable Decrypt/Encrypt **一次性复制**后切断外链 (I/O / launch / SIM 单库语义与 device 探针一致; 探针 `STABLE*_ROOT` 仅为测试便利)。

目录

1 工程约束（自包含）	3
1.1 允许的外部依赖	3
1.2 禁止的外部依赖与不安全路径	3
1.3 相对 device 探针的落点差异	3
1.4 唯一交付路径（默认即全量）	4
1.5 目录布局（计划；写码阶段落地）	4
2 数学语义（Alg.21 → Alg.18 → Alg.15/14）	5
2.1 参数集	5
2.2 Algorithm 21（对外）与 Alg.18（设备 I/O）	5
2.3 相对 PKE Decrypt / Encaps 的增量	5
2.4 liboqs 对拍口径	6
2.5 Alg.15 / Alg.14 主体	6
3 编排与 Launch	6
3.1 SIM（生产路径；锁定 4 launch）	6
3.2 CPU（tikicpu 分叉；锁定 6 launch）	6
3.3 SIM 单库与同名头隔离（强制）	7
3.4 相对 correctness（vendor G4+G5）	7
4 AI Core 规模	7
5 GM 布局与头段 / FO 同步	8
5.1 KemDecPhaseEHead（仅 block0）	8
5.2 KemDecFo（设备 FO）	8
5.3 双 AIV 与头段次序（强制）	8
6 踩坑与强制条款（换 Agent 必读）	9
7 全量 API 清单（查阅 CANN 9.0.0）	9
7.1 设备哈希（非 CANN 矢量 API；工程共享库）	10
7.2 评估过但未采用	10
8 数学步骤 → API 覆盖率	11
9 验证	12
9.1 Golden 角色	12
9.2 生产验收（默认；写码完成后）	12
9.3 Gate 建议（写码阶段，非本规格交付物）	12
10 性能（SIM 910B4 参考）	12
11 写码衔接（本规格确认后）	12

1 工程约束（自包含）

1.1 允许的外部依赖

路径	用途
library/shared/shake_x of_kernel/	Prep SampleNTT / PRF (Phase-E)
library/shared/keccak_ f1600_kernel/	Keccak-f[1600]: Phase-E 头 Sha3OneShot (SHA3-512: $G(m' h)$); FO 尾 Shake256OneShot ($J(z c)$)
library/shared/f203_un ified_round/	Encrypt Compress / Decrypt Decompress 统一整 数向量 (随 vendor)
library/shared/	公共头 (shake_general.h、 fips203_device_sha3.hpp 等)
CANN / tikicpulib	工具链与仿真 (非业务源码)

1.2 禁止的外部依赖与不安全路径

- 运行时 / 编译期依赖 ascendc-tests/ (含 pass-fix-f203-alg21-kem-decaps-device-k4 /、correctness)、frozen/、其它 examples/*/ (一次性 vendor 除外)。
- 子进程调用 stable / 探针 run.sh 再拼 Decaps。
- Host tiny_sha3 / liboqs 参与生产路径的 G/J/密文比对选 K (KAT / golden 脚本除外)。
- 从 ascendc-tests/frozen/ 或 correctness vendor/ (G4/G5) 抄实现 / vendor_sync。
- Host 预填 r' ; 将 $m'/c'/K'$ 落盘为交付物; Host FO 写 K.bin。

1.3 相对 device 探针的落点差异

项	device 探针	本 exp (锁定)
Decrypt / Encrypt 源码	CMake STABLE_*_ROOT; SIM 用 prepare_dec_shim.sh 合 库	本目录 vendored Decrypt+Encrypt; 冲突同 名头在 vendor 时改名 (如 dec_*) 或等价隔离
KEM 增量	kem/ (G / FO)	同语义; vendored 于 kem/
Launch / I/O / GM SIM session	SIM 4 / CPU 6; 仅 K 默认 decaps_1session	同契约 同 ; 对照 decaps_2session 标非默 认
SIM tick 参考	D286803+E745925	写码目标同量级

1.4 唯一交付路径（默认即全量）

组件	锁定配置
Phase-D	f203_decrypt_device_fused: 自 dk_kem 指针切 dk _{PKE} /c → m' (行 1-5 并入 Decrypt 入口; 不另开 launch)
Phase-E 头	Launch prep 前段: KemDecPhaseEHead (仅 block0): m' _{GM} h → G → K' r'
Prep $\hat{A}$ / CBD	与 stable Encrypt: F203_AHAT16_BLOCK_DIM=2; 双 AIV 分片 SampleNTT; PRF(r', ·)+CBD _{η=2}
Compute+pack	与 stable Encrypt: SIM 内联 f203_encrypt_118_119 写出 c'; NTT S1-S3 禁 Gather
FO	设备: c $\stackrel{?}{=}$ c' 则 K ← K', 否则 K ← J(z c); 首版 SIM 允许过渡 fo_only; CPU 用 pack_fo
Host	SIM 4 / CPU 6; dk_kem+c+LUT → 仅 K
宏默认	与 Decrypt/Encrypt 生产全量一致; 禁止关 pack / 关向量路径作默认验收

禁止将「Host 注入 r'」「Host FO」「关 ByteEncode」「分段只验 Phase-E」等作为默认 run.sh 验收; 调试对照 (如 KEM_DECAPS_PHASEE_ONLY=1、KEM_DECAPS_REJECT=1、decaps_2session) 须显式覆盖 env, 并在头注释标「非默认」。

1.5 目录布局（计划; 写码阶段落地）

```

stable-fips203-mlkem-kem-decaps-k4/
  stable-fips203-mlkem-kem-decaps-k4-实现方案-customspec.tex/.pdf
  main_kem_decaps.cpp           # Host: SIM 4 / CPU 6; D2H 仅 K
  f203_kem_dec_layout.h         # I/O 常量 (3168/1568/32)
  kem/
    f203_kem_dec_phase_e_init.hpp # KemDecPhaseEHead (G)
    f203_kem_dec_fo.hpp           # KemDecFo (比对 + J)
    f203_kem_dec_phase_e_prep_entry.cpp
    f203_kem_dec_fo_entry.cpp     # SIM fo_only / CPU pack_fo 入口
  decrypt/                       # vendored Alg.15 (可含 dec_* 改名)
  encrypt/ 或 prep/+compute/     # vendored Alg.14
  cmake/decaps/CMakeLists.txt   # 单 ascendc_library (CPU/SIM)
  scripts/                      # gen_data / verify / KAT 胶水;
                                # 可选 prepare_dec_shim 或写码时一次性改名
  run.sh
  SELF_CONTAINED.md             # 写码阶段补

```

STATUS.md

写码验收后补

vendor 来源 (一次性, 写码时复制后切断):

- examples/stable/stable-fips203-mlkem-pke-decrypt-k4/ (Phase-D 可达树; 冲突头改名隔离)。
- examples/stable/stable-fips203-mlkem-pke-encrypt-k4/ (Phase-E Encrypt 主体)。
- KEM 增量对照 (只读行为, 禁止编译依赖) ascendc-tests/pass-fix-f203-alg21-kem-decaps-device-k4/kem/。

2 数学语义 (Alg.21 → Alg.18 → Alg.15/14)

2.1 参数集

ml_kem_1024: $n = 256$, $q = 3329$, $k = 4$; $d_u = 11$, $d_v = 5$; $\eta = 2$ 。(与本仓 PKE Decrypt/Encrypt / KEM Encaps 交付同档; 勿与 ML-KEM-768 混用。)

2.2 Algorithm 21 (对外) 与 Alg.18 (设备 I/O)

对外 API 可对应 Alg.21。本用例设备核锁定 Alg.18 形态 (与 pass-fix 一致):

$dk \in \mathbb{B}^{3168}$, $c \in \mathbb{B}^{1568}$	Host H2D: dk_kem 、 c
$dk = dk_{PKE} ek h z$	$ dk_{PKE} =1536$, $ ek =1568$, $ h = z =32$
$m' \leftarrow K\text{-PKE.Decrypt}(dk_{PKE}, c)$	Alg.15: Phase-D
$(K' r') \leftarrow G(m' h) = \text{SHA3-512}(m' h)$	Phase-E prep 前段
$c' \leftarrow K\text{-PKE.Encrypt}(ek, m', r')$	Alg.14: 随机性输入即为 r'
$K \leftarrow \begin{cases} K' & \text{若 } c = c' \\ J(z c) & \text{否则 (SHAKE256, } \ell=32) \end{cases}$	设备 FO
输出 $K \in \mathbb{B}^{32}$	

- h 、 z 、 ek 、 dk_{PKE} : 自 dk_kem 指针偏移切片; 行 1-4 不另开 launch。
- r' : 仅 device 写 workspace; prep 以 r' 为种子做 PRF+CBD; 不落盘为交付物。
- m' 、 c' 、 K' : workspace; 生产路径不 D2H。

2.3 相对 PKE Decrypt / Encaps 的增量

项	PKE Decrypt / Encaps	本 KEM Decaps
生产输入	Decrypt: $dk_{PKE}+c$; Encaps: $ek+m$	$dk_{kem}+c$ (r' 非 Host 输入)

共享秘密 K	Encaps: G 前半直出	FO 后选出 (合法 = K' ; 拒绝 = J)
密文	Decrypt 消费 c ; Encaps 产出 c	消费 c ; 内部再生 c' 仅供比对
Launch 数	Decrypt SIM 1; Encaps SIM 2	SIM 4 / CPU 6 (含过渡 FO)

2.4 liboqs 对拍口径

验收对齐 OQS_KEM_ml_kem_1024 的 decaps:

```
input:  dk_kem[3168], c[1568]
output: K[32]      # 逐字节 == liboqs (合法与拒绝路径均须)
```

Golden / KAT 仅验 I/O 等价; 禁止以「与 device / correctness 源码同构」验收。拒绝路径 (假密文) 期望 $K = J(z||c)$, 与 liboqs Decaps 一致。

2.5 Alg.15 / Alg.14 主体

Phase-D 与 exp-fips203-mlkem-pke-decrypt-k4-实现方案-customspec.tex、stable Decrypt 同构; Phase-E Encrypt 段与 exp-fips203-mlkem-pke-encrypt-k4-实现方案-customspec.tex 同构 (本文不重复展开公式)。本阶段不做: D+E 单 launch 融合、为头段单独开产品化核名、改 PKE 数学。

3 编排与 Launch

3.1 SIM (生产路径; 锁定 4 launch)

```
aclInit / CreateStream      # 默认单 session (decaps_1session)
Launch-D: f203_decrypt_device_fused    // MIX; m'_gm
Launch-E1: f203_kem_dec_phase_e_prep    // AIV_ONLY, blockDim=2
        block0: KemDecPhaseEHead(m',h → K'_gm, r'_gm) // G
        dual AIV: BuildEncryptPrepSinglePipe          // A-hat + CBD <- r'
Launch-E2: f203_encrypt_l18_l19         // MIX; 写出 c'
Launch-E3: f203_kem_dec_fo_only          // AIV_ONLY; FO → K_gm
        (过渡; 目标收回 E2 尾)
aclrtSynchronizeStream
D2H: K.bin (32)    # 禁止默认 D2H m'/c'/K'
aclFinalize
```

3.2 CPU (tikicpu 分叉; 锁定 6 launch)

```
#1 decrypt_device_fused
#2 f203_kem_dec_phase_e_prep
```

```
#3 ntt_y (同 Encrypt CPU)
#4 at_jp
#5 intt_e1
#6 pack_fo (Encrypt pack + 设备 FO; 替代纯 pack)
```

CPU 路径**不是**与 SIM 同构的完整设备拓扑;验收仍以 $K_{\max}=0$ 为准。须用 ASCENDC_BUILD_CPU/ASCENDC_BUILD_SIM + ASCENDC_SIM_HOST_MODE, 禁止 per-probe 宏分叉破坏单库。

3.3 SIM 单库与同名头隔离 (强制)

Decrypt 与 Encrypt 树存在同名头时, ascendc precompile **无法** per-TU 隔离。本 exp 写码须在 vendor 时完成隔离(推荐:Decrypt 冲突 basename $\rightarrow$ dec_* 并改写 #include;或等价 shim 生成后纳入本目录且**不**依赖仓库外 stable 路径作长期编译根)。CPU/SIM **均为单** libascendc_kernels_*.so。默认 ASCENDC_SIM_HOST_MODE=decaps_1session; decaps_2session 仅作对照 (非默认)。

3.4 相对 correctness (vendor G4+G5)

项	correctness-k4	本方案 / pass-fix
Decrypt/Encrypt G/FO	frozen G4/G5 vendor 常与 oracle 拼装	stable 对齐 / vendored 设备 SHA3/SHAKE; FO 设备完成
r'	可能 Host/oracle	仅 device G
SIM	历史双库/多 session	单库 +1session

4 AI Core 规模

段	核类型 / blockDim / 角色
Phase-D	与 stable Decrypt fused: MIX; blockDim 对齐 Decrypt 交付 (通常 aicore=1 量级); 入口前段切 dk_kem
Phase-E prep	KERNEL_TYPE_AIV_ONLY; blockDim=2; 双 AIV 各 8 poly 分片 $\hat{A}$; 仅 block0 执行 KemDecPhaseEHead 与 PRF+CBD
Phase-E compute	KERNEL_TYPE_MIX_AIC_1_2; blockDim=1; 1 AIC + 2 AIV; NTT/INTT/内积/内联 pack $\rightarrow c'$
FO (过渡)	KERNEL_TYPE_AIV_ONLY; blockDim=1 (或与 pass-fix 一致); 仅 block0 做比对 + J 选路写 K
KEM 头 G	不 新增独立产品化 launch; 并入 prep 前段

内部命名: prep aiv=2; compute aicore=1 (MIX 1:2); FO 过渡 aiv=1。**选型理由:** 复用已验收 Decrypt/Encrypt/Encaps 拓扑; pass-fix 已证明 SIM 4 / CPU 6 全链绿。**禁止**实现擅自改 blockDim 却不回写本规格。

串行占用: Phase-D 一颗 MIX AI Core; Phase-E prep 2 Vector; compute 1 MIX; FO 1 Vector。
CPU [SUCCESS][AIC_*] 为 tikicpu artifact; 以 SIM profile_*.toml / Total tick 为准。

5 GM 布局与头段 / FO 同步

GM 缓冲	尺寸	用途
dk_kem_gm	3168	H2D; 切 dk _{PKE} /ek/h/z
c_gm	1568	H2D; Decrypt 入 + FO 比对左端
m_prime_gm	32	Phase-D 写 / Phase-E 读; 不 D2H
K_prime_gm	32	G 前半; FO 合法分支
r_prime_gm	32	G 后半; =coins; prep CBD 种子; 不 D2H
$\hat{A} / (y\ e_1\ e_2)$	同 Encrypt	prep 写 / compute 读
c_prime_gm	1568	Encrypt 写; FO 比对右端; 不默认 D2H
K_gm	32	FO 写; 唯一交付 GM
ws+LUT	同 D/E	Decrypt/Encrypt workspace

不新增交付契约: m_prime.bin、c_prime.bin、Host 侧 r' 输入文件。

5.1 KemDecPhaseEHead (仅 block0)

1. 自 m_prime_gm 读 $m'[32]$ UB。
2. 自 dk_kem 偏移读 $h[32]$ (或入口已切片指针)。
3. 拼 $m'\|h$ (64 B) $\rightarrow$ Sha30neShot (mdlen=64) $\rightarrow K'\|r'$ 。
4. 写 K_prime_gm、 $r' \rightarrow r_prime_gm$ 。

5.2 KemDecFo (设备 FO)

1. 逐字节 (或向量化) 比较 c 与 c' (长度 1568)。
2. 相等: $K \leftarrow K'$; 否则 $K \leftarrow J(z\|c)$ (Shake256OneShot, 输出 32 B)。
3. 写 K_gm。

禁止把「恒选 K' 」或「Host 比对后写 K 」当作生产路径。

5.3 双 AIV 与头段次序 (强制)

- SIM/NPU: GetBlockIdx()==0 先完成 KemDecPhaseEHead, 再与 block1 并行跑 BuildEncryptPrepSingleP
SampleNTT 只依赖 ek 尾 ρ , 不依赖 r' ; CBD/PRF 仅 block0, 故须保证 block0 在 CBD 前
已写完 r_prime_gm。
- CPU (ASCENDC_CPU_DEBUG 且 blockDim=2): 头只跑一次。

- Phase-D→Phase-E: 同 session 下须保证 m' GM 对后续 launch 可见 (stream 序 + 必要 sync; 对齐 pass-fix decaps_1session)。

6 踩坑与强制条款（换 Agent 必读）

条款	要求
禁 Host 伪 FO	禁止 Host SHA3/memcmp 写 K 冒充 Decaps
禁预填 r'	生产 <code>gen_data</code> 不得提供 Host 侧 r' 输入契约
禁 G4/G5 / frozen	禁止 <code>vendor_sync</code> frozen; 禁抄 frozen 源码
禁独立 G launch	G 必须并入 Phase-E prep; 否决 correctness 式独立头核
禁长期编译依赖探针/stable	device 只读行为; incubating 须 vendored 自包含
禁双库默认 SIM	须单 <code>libascendc_kernels_sim.so</code> + 默认 1session
同名头	Decrypt/Encrypt 冲突头必须改名隔离, 否则 SIM 合库失败
已晋级 stable	2026-07-20 # 交付 # 自 incubating 复制; 预研副本保留
中间态落盘	$m'/c'/K'$ 默认禁作交付; 调试 dump 标非默认
run.sh 坑	不得在 <code>source env.sh/setenv</code> 前 <code>set -e</code> ; 防挂死预算建议 $\geq 600s$ (全链可 1200)
cwd	二进制须在用例根 cwd 跑 (相对路径 <code>./input/...</code>)

7 全量 API 清单（查阅 CANN 9.0.0）

说明: Decrypt / Encrypt 主体 API 与对应 stable / exp customs spec 同集, 已在 `library/documents/CANN-AscendC算子开发接口参考-查阅索引.md` 有记录; 下表列本用例须显式锁定的主路径 API (含 KEM 头与 FO)。分类按 PDF 第 2 章; 在线 ID 为社区 `atlasascendc_api_07_*`。

API	分类	在线 ID	A2	输入/输出 (本用例)	备注
<code>GlobalTensor</code>	2.2	07_0007	✓	GM 描述	<code>SetGlobalBuffer</code>
<code>LocalTensor</code>	2.2	07_0006	✓	UB 描述	D/E 向量段
<code>GetBlockIdx</code>	2.3 系统	07_0185	✓	核索引	头/FO 仅 block0; prep 分片
<code>GetSubBlockIdx</code>	2.3 系统	07_0186 邻	✓	AIC/AIV 子块	MIX
<code>KERNEL_TYPE_AIV_ONLY</code>	Utils	编程指南	✓	prep / FO	
<code>KERNEL_TYPE_MIX_AIC_1_2</code>	Utils	编程指南	✓	Decrypt fused / Encrypt L2	

API	分类	在线 ID	A2	输入/输出 (本用例)	备注
TPipe/TQue/TBuf	2.3 资源	07_0108/0137	✓	管道	D/E
DataCopy	2.3.1	07_0101	✓	uint8/int32 GM↔UB	头/FO/主路径
DataCopyPad	2.3.1	07_0265	✓	可选跨步	头/FO 主路径不用
PipeBarrier	同步	常用	✓	本核 MTE/V	头写 r' 后可见性 (若需)
CrossCoreSetFlag	同步	API 列表	✓	AIC↔AIV	Decrypt/Encrypt FSM
CrossCoreWaitFlag	同步	API 列表	✓	同上	与 stable 同构
ShiftRight/Muls/Adds/AddSub	2.3.3	07_0059/55/54/35/36	int32		D/E 向量链
Duplicate	2.3.3	07_0041 邻	✓	填充	Decrypt pad / Prep
GatherMask	2.3.3.4	07_0071	✓	掩码收集	rej compact (非 NTT S1-S3)
Gather	2.3.3	07_0071 邻	✓	—	NTT S1-S3 禁用 ; Encode 后另议
Cube MMAD / AicMmad	2.3.2	教程 + 封装	✓	int8×int8→int32	Stage2

7.1 设备哈希 (非 CANN 矢量 API; 工程共享库)

符号	契约
F203SeDeviceKeccak:: Sha30neShot	library/shared/keccak_f1600_kernel/fips203_device_sha3.hpp; mdlen=64: $G(m' h) \rightarrow K' r'$ 。勿改拼接顺序与切分。
F203SeDeviceKeccak:: Shake2560neShot (或 等价 J)	同共享库; $J(z c) \rightarrow 32\text{B}$; 拒绝路径 FO 必用。

7.2 评估过但未采用

方案	结论
独立 launch 仅算 G	多 sync; pass-fix 已否决; 禁止
Host 预填 r' + 纯 Encrypt	非 Alg.18 设备 Decaps; 禁止 作生产

Host FO	伪设备；禁止
(memcmp+J) 写 K	
frozen / correctness	路线关闭；禁止抄码
G4+G5	
编译期长期	incubating 须自包含 vendor；探针例外
STABLE_*_ROOT	
SIM 双库 + 默认	T2 已淘汰作默认；对照可保留非默认
2session	
NTT S1-S3 内	Tag5T 禁令；沿用 Encrypt
Gather	
未绿即复制 stable	禁止
Decaps	
首版强制 SIM 3	工程债可选；非首版门禁
launch (FO 内联)	

8 数学步骤 → API 覆盖率

数学步骤	主路径	覆盖	缺口/备注
切 dk_kem	指针偏移	100%	不另开核
		Host/入口	
Decrypt→ m'	MIX 向量 +Cube	高	同 stable Decrypt
G(m' h) → K' r'	SHA3-512 共享库	标量	写 K'/r' GM
SampleNTT Â	SHAKE + rej 向量	部分	同 Encrypt Prep
PRF+CBD _{η=2}	向量/标量混合	高	种子改自设备 G
NTT / 内积 /	MIX 向量 +Cube	高	S1-S3 无 Gather
INTT→ c'			
c ≐ c'	标量/小块搬运	标量	1568 B 比对
J(z c)	SHAKE256 共享库	标量	拒绝路径
写出 K	标量/DataCopy	100%	32 B
		搬运	

主路径结论：Decrypt/Encrypt 计算段与 stable 同覆盖率；KEM 增量哈希与 FO 为显式标量缺口（设备共享 Keccak，非矢量 API）。

9 验证

9.1 Golden 角色

Host / liboqs 仅提供合法输入与期望输出（黑盒 oracle）。**禁止**以「与 device / correctness 源码同构」为验收；仅 I/O 等价。

9.2 生产验收（默认；写码完成后）

```
cd examples/stable/stable-fips203-mlkem-kem-decaps-k4
bash run.sh -r cpu -v Ascend910B4
bash run.sh -r sim -v Ascend910B4 # 默认全量 + 单库 + 1session
# 检查 output/K.bin (32B); max=0; 用例根无 stray dump
```

对照（写码阶段）：同 dk/c 下与 pass-fix-f203-alg21-kem-decaps-device-k4 及/或 liboqs decaps 的 K 逐字节一致。建议分项：合法路径 + KEM_DECAPS_REJECT=1 拒绝路径（均标清默认/非默认）。

禁止把一长串与 default 相同的 HAT_*=0/1 写入标准验收命令。

9.3 Gate 建议（写码阶段，非本规格交付物）

Gate	验收
G0	CMake+run.sh 壳；单库；kernel 结束
G1	Phase-E-only（非默认）：固定 $m'/h/z/c$ ； K vs oracle
G2	全链合法路径： K vs liboqs；CPU max=0
G3	SIM 同 G2；单库 + 1session；更新 tick；根无 stray dump
G4	拒绝路径： K vs liboqs Decaps ($\equiv J$)；CPU+SIM
G5	默认 run.sh CPU+SIM PASS；分项 KAT 建议 CPU×10+SIM×3

10 性能（SIM 910B4 参考）

- pass-fix 全链：D Total tick **286803** + E **745925**（合法路径； K max=0）。
- Phase-E-only 历史参考 tick $\approx$ **746221**（E2）。
- 本 exp 写码验收目标：与 pass-fix 同量级；**远超**（无日志挂死）视为回归。
- ~15s 仅为 Tag5T NTT 全流程性能定标，不套用本用例防挂死预算；预算写进本用例 run.sh（建议默认 ≥ 600 s，全链可 1200）。

11 写码衔接（本规格确认后）

- 用户确认本 customspec（含 §6、I/O、Launch、自包含、SIM 4 含过渡 fo_only）。

2. 用户明确 **【预研】** / 「可以写代码」并指明本路径后，按 `.cursor/skills/pre-research/S KILL.md` 在本目录落地 vendored 源码（当前目录**无**实现可抄；device 仅对照行为）。
3. Golden: liboqs / pass-fix 对照；登记表若需 Decaps 专表则另开 docs/specs/（缺项须停下请用户补来源）。
4. incubating 经验收阶梯稳定后，用户 # 交付 # 再**复制**晋级 examples/stable/stable-fips203-mlkem-kem-decaps-k4/（名称待交付时确认）；**禁止**未绿先晋级。